Automata Theory

Turing Machines Variants

CS32022

By Thisara Watawana

13 October 2026

9. Turing Machines Variants

Variant 1: Multitape Turing Machines (*k*-Tape TMs)

A Multitape Turing Machine consists of a finite control connected to k distinct, infinite tapes. Each tape has its own read/write head that can move independently left, right, or stay put (S).

- **Formal Transition Function:** Instead of reading and writing a single symbol, the transition reads a tuple of k symbols and writes a tuple of k symbols:

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

- **Example Rule:** $\delta(q_0, a, \sqcup) = (q_1, X, b, R, L)$
  (In state q_0, if tape 1 sees 'a' and tape 2 sees '$\sqcup$', switch to state q_1, replace tape 1's 'a' with 'X' and move right, replace tape 2's '$\sqcup$' with 'b' and move left).

9. Turing Machines Variants

Variant 1: Multitape Turing Machines (k-Tape TMs)

- **Simulation Theorem:** Every *k*-tape Turing Machine *M* has an equivalent 1-tape Turing Machine *S*.
- **How a 1-Tape TM Simulates *k* Tapes:** The single-tape simulator *S* represents all *k* tapes on one single tape by separating them with a special boundary symbol (#). To track where the *k* independent tape heads are currently located, *S* uses dotted/marked versions of tape symbols (ȧ, ḃ, ␣̇).
- **Multitape TMs run faster** ($O(n)$ time vs $O(n^2)$ on a 1-tape TM for problems like string copying/palindromes), but they do not recognize any new languages. They only improve operational efficiency.

9. Turing Machines Variants

Exercise 01

Target Language: $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Exercise Objectives:

- Design a 2-Tape Deterministic Turing Machine (M_2) that decides L.
- Design a standard 1-Tape Deterministic Turing Machine (M_1) that decides L.
- Compare the time complexity and design mechanics of both approaches.

9. Turing Machines Variants

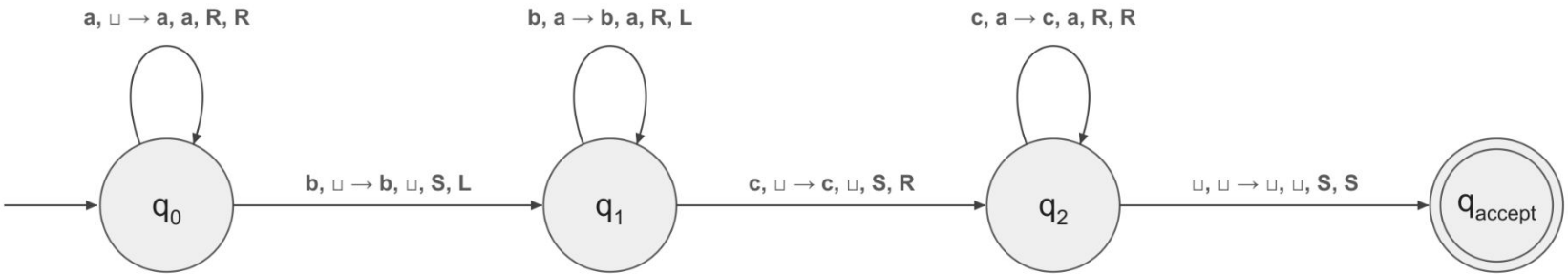
Exercise 01: Solution

Instead of erasing or marking anything, Tape 2 acts as a reusable ruler:

- **Phase 1 (Copy *a*'s):** Read *a*'s on Tape 1 and copy them onto Tape 2, moving both heads Right (*R*, *R*).
- **Phase 2 (Match *b*'s):** When Tape 1 hits *b*, shift Tape 2 left by one cell (*S*, *L*). Then read *b*'s on Tape 1 while walking Tape 2 backward (Left) over the *a*'s. When Tape 2 hits the blank to the left of the *a*'s, you have verified #*a* = #*b*.
- **Phase 3 (Match *c*'s):** Shift Tape 2 right by one cell (*S*, *R*). Then read *c*'s on Tape 1 while walking Tape 2 forward (Right) over the exact same *a*'s. When Tape 1 hits ⊔ and Tape 2 hits ⊔ simultaneously, you have verified #*a* = #*c*.

9. Turing Machines Variants

Exercise 01: Solution



9. Turing Machines Variants

Exercise 01: Solution

Metric	1-Tape TM (M_1)	2-Tape TM (M_2)
Language Decidability ($a^n b^n c^n$)	Decidable ($L \in R$)	Decidable ($L \in R$)
Time Complexity	$O(n^2)$ (Quadratic time)	$O(n)$ (Linear time)
Active States (Excluding Halting)	5 states (q_0, q_1, q_2, q_3, q_4)	3 states (q_0, q_1, q_2)
Auxiliary Tape Markers	3 replacement markers (Γ requires X, Y, Z)	Zero markers (Γ only requires {a, b, c, ␣})
Head Movement Pattern	Continuous back-and-forth shuttling (n round trips across input)	Single-pass sequential sweeps (no zigzagging)
Tape Manipulation	Destructively overwrites input symbols cell-by-cell	Non-destructive: keeps input on Tape 1 intact while using Tape 2 as a temporary count ruler
Simulation Cost	Base execution model	Requires $O(t^2)$ steps when converted down to a 1-tape simulator

9. Turing Machines Variants

Variant 2: Nondeterministic Turing Machines (NTMs)

In an NTM, at any step, the machine can have multiple possible next moves. The computation branches into a tree of configurations.

- **Formal Transition Function:** The transition function maps a state and symbol pair to a set of possible choices:

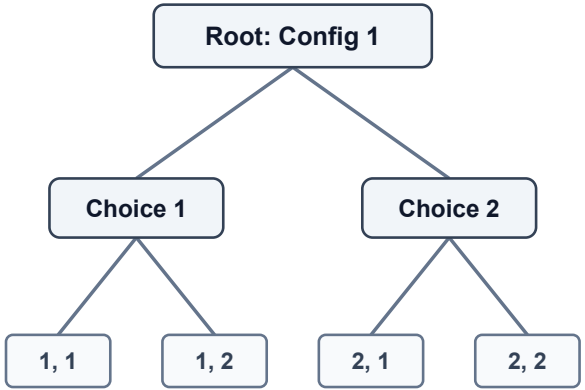
$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

- An NTM accepts an input string w if at least one branch in its computation tree reaches q_{accept}.

9. Turing Machines Variants

Variant 2: Nondeterministic Turing Machines (NTMs)

- **Simulation Theorem:** Every Nondeterministic Turing Machine *N* has an equivalent Deterministic Turing Machine *D*.
- **How a Deterministic TM Simulates an NTM:** The deterministic simulator *D* explores the computation tree of *N* using a 3-Tape Deterministic TM:
    - **Tape 1 (Input Tape):** Holds original input string *w* (read-only master copy).
    - **Tape 2 (Simulation Tape):** Working copy of *N*'s tape for running a single branch of computation.
    - **Tape 3 (Address Tape):** Tracks *D*'s location in *N*'s computation tree (e.g., "231" = choice 2 at step 1, choice 3 at step 2, choice 1 at step 3).



9. Turing Machines Variants

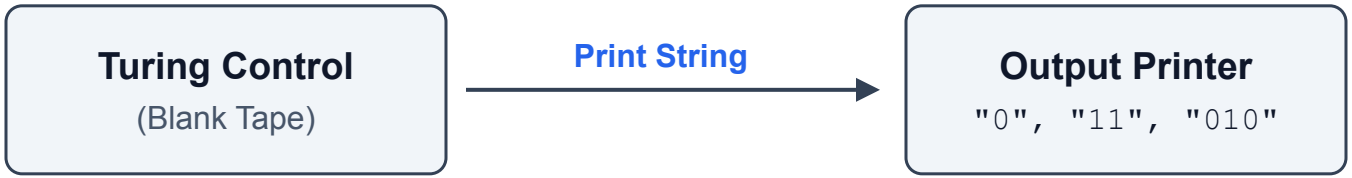
Variant 2: Nondeterministic Turing Machines (NTMs)

- **The Critical Algorithm Choice: BFS vs. DFS**
  - **Why Depth-First Search (DFS) FAILS:** If *D* uses DFS, it could follow an infinite non-accepting branch forever, missing an accepting state sitting just 2 steps away on another branch.
  - **Why Breadth-First Search (BFS) WORKS:** *D* generates addresses in order of length (1, 2, …, *n*, then 11, 12, …). BFS guarantees that if an accepting configuration exists at finite depth, *D* will eventually find it and halt.
- **Nondeterminism allows exponential parallel search**, but it does not expand decidability. Every NTM language is still decidable/recognizable by a deterministic TM. This equivalence forms the exact foundation for ***P*** vs ***NP***.

9. Turing Machines Variants

Variant 3: Enumerators

An Enumerator is a Turing Machine equipped with an attached printer output device. It starts with a blank tape and continuously runs, printing out strings one by one.



The Equivalence Theorem

Theorem: A language *L* is Turing-recognizable if and only if some Enumerator enumerates *L*.

9. Turing Machines Variants

Variant 3: Enumerators

Equivalence Logic (Directional Breakdown)

⇒ **Direction** (If *L* is Recognizable, an Enumerator exists):

Let *M* recognize *L*, and let s_1, s_2, s_3, … be all possible strings in Σ*. To handle infinite loops, the Enumerator runs *M* using dovetailing:

- **Step 1:** Run *M* on s_1 for 1 step.
- **Step 2:** Run *M* on s_1, s_2 for 2 steps.
- **Step *i*:** Run *M* on s_1, s_2, …, s_i for *i* steps.

Whenever *M* accepts a string s□, print s□.

9. Turing Machines Variants

Variant 3: Enumerators

Equivalence Logic (Directional Breakdown)

⇐ **Direction** (If an Enumerator exists, a Recognizer TM *M* exists):

Given input *w*, *M* runs the Enumerator *E*. As *E* prints strings s_1, s_2, …, *M* compares each printed string to *w*. If $w = s_i$, *M* halts and **ACCEPTS**.

9. Turing Machines Variants

TM Variant	Architectural Feature	Equivalent Model	Simulation Technique	Complexity Impact
Multitape TM	k Tapes, k heads	Standard 1-Tape TM	Multi-symbol tape with delimiters (#) and dotted head markers ($\dot{a}$)	Quadratic time slowdown ($O(n^2)$)
Nondeterministic TM	Multi-branch choices per state	Standard 1-Tape TM	3-Tape BFS exploration of the computation tree	Exponential time slowdown ($O(2^n)$)
Enumerator	Tape control + Printer output	Recognizer TM	Dovetailing simulation over input strings Σ^*	N/A (Functional equivalence)

9. Turing Machines Variants

Encoding a Machine as Data (⟨M⟩)

The Core Premise: Every component of a Turing Machine $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ is finite. Therefore, any TM can be serialized into a plain binary text string.

A Simple Binary Encoding Scheme:

- **States: q_1, q_2, q_3, … ⇒ 1, 11, 111, …**
- **Tape Symbols: 0, 1, ␣ ⇒ 1, 11, 111**
- **Directions: L, R ⇒ 1, 11**

9. Turing Machines Variants

Encoding a Machine as Data (⟨M⟩)

Encoding a Transition Rule: The rule $\delta(q_1, 0) = (q_2, 1, R)$ becomes a string separated by zeros:

Code = 1 0 1 0 11 0 11 0 11

Encoding the Full Machine ⟨M⟩: Chain all transition codes together separated by double zeros (00).

Key Conclusion:

Any Turing Machine *M* ⇔ A finite binary string ⟨M⟩ ∈ {0, 1}*

9. Turing Machines Variants

The Universal Turing Machine (UTM)

A Universal Turing Machine (U) is a single, fixed Turing Machine that takes two inputs on its tape: a program description ⟨M⟩ and an input string w.

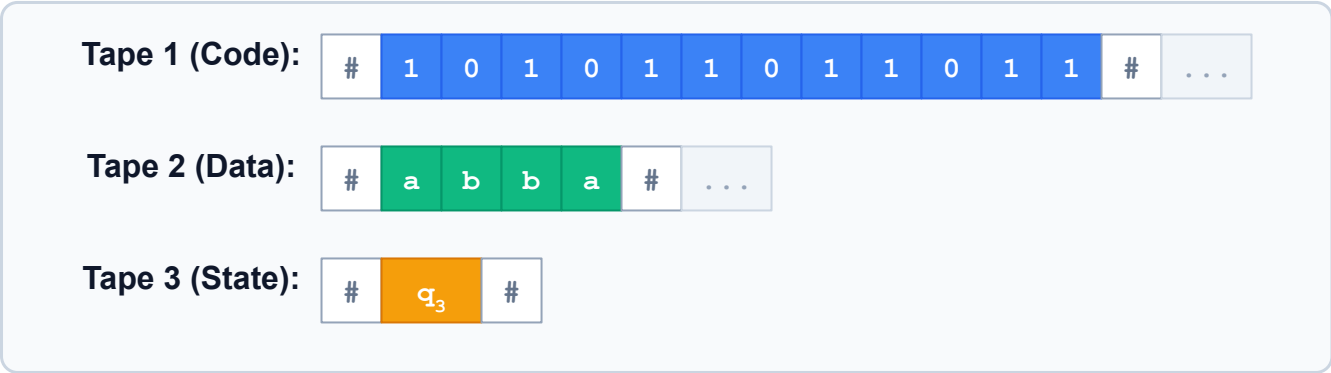
$$\mathbf{U(\langle M, w \rangle) = \text{Simulates M on w}}$$

How U Operates (3-Tape View):

- **Tape 1:** Stores the code of M (⟨M⟩).
- **Tape 2:** Stores M's virtual tape (initialized with w).
- **Tape 3:** Stores M's current state.

9. Turing Machines Variants

The Universal Turing Machine (UTM)



The Simulation Cycle:

1. Look at *M*'s current state on Tape 3 and the current symbol on Tape 2.
2. Scan Tape 1 to find the matching transition rule in $\langle M \rangle$.
3. Update Tape 2 (write symbol, move head) and Tape 3 (update state).
4. If *M* enters q_{accept}, *U* **accepts**. If *M* enters q_{reject}, *U* **rejects**.

9. Turing Machines Variants

The Birth of Software

Before Alan Turing (1936): Calculators were single-purpose hardware. If you wanted a machine to calculate trajectories, you built Machine A. If you wanted to compute mortgage payments, you physically built Machine B.

Turing's Insight (The UTM): You don't need different hardware machines for different tasks. You build ONE single hardware machine (U) that reads a description of another machine ($\langle M \rangle$) from memory and simulates it.

$$\text{Universal Turing Machine (U)} \Leftrightarrow \text{Operating System / CPU}$$

$$\text{Machine Encoding } (\langle M \rangle) \Leftrightarrow \text{Software / Python Script / .exe file}$$

$$\text{Input } w \Leftrightarrow \text{Data File / User Input}$$

Questions?

thisaraw@boffin.lk